Автономная некоммерческая организация профессионального образования
КАЛИНИНГРАДСКИЙ БИЗНЕС-КОЛЛЕДЖ

Кафедра Информационных технологий

Курсовая работа

На тему: **«Создание системы «Команда разработчиков»»**

По МДК **Технология разработки программного обеспечения**

Выполнила студентка
группы 24-ИСП-4/1
Сапунова С. В.

Проверил:
Парамонов Д.Д.

Оценка:

_____

Калининград
2026

Содержание

Введение 3
1. Разработка диаграммы вариантов использования 4
2. Разработка технического задания 5
3. Разработка диаграммы деятельности 11
4. Разработка диаграммы взаимодействий 12
5. Модели и генерирование кода на основе UML-диаграмм 13
6. Оптимизация программного кода 15
7. Разработка программного обеспечения для проектируемой предметной области 18
8. Работа с системой контроля версий 23
Заключение 24
Список использованных источников 25
Список сокращений 26
Приложение 1 27
Приложение 2 28

Введение

Актуальность. Взаимодействие заказчика и команды разработки часто сопровождается потерей информации, технические требования передаются в устной форме или в виде разрозненных документов, в которых легко запутаться или упустить что-то из виду. В результате исполнители трактуют задачу иначе, чем предполагал заказчик, это приводит к переделкам, сдвигу сроков и росту стоимости проекта.

Система «Команда разработчиков» формализует этот процесс: она фиксирует требования в структурированном виде и связывает их с конкретными задачами, сроками и бюджетом. Заказчик видит, за что платит, менеджер проекта контролирует нагрузку специалистов, разработчик получает четкие инструкции и может фиксировать фактически затраченное время.

Такой подход снижает количество спорных ситуаций и делает оценку трудозатрат прозрачной.

Цель работы – создать систему «Команда разработчиков».

Задачи:

1. Спроектировать диаграмму вариантов использования;
2. Разработать техническое задание;
3. Создать диаграмму деятельности;
4. Составить диаграмму взаимодействия;
5. Сгенерировать модели и код на основе UML-диаграмм;
6. Оптимизировать программный код;
7. Разработать программное обеспечение для проектируемой предметной области;
8. Реализовать использование системы контроля версий для системы;

1. Разработка диаграммы вариантов использования

Диаграмма вариантов использования – диаграмма, на которой отображаются функции проектируемой системы, внешние действующие лица и отношения между ними.

Диаграмма вариантов использования предназначена для достижения следующих целей:

1. Определить набор функций, действующих лиц и их взаимодействия;
2. Указать требования к отдельным функциям в виде набора действий;
3. Разработать базовую модель программы для ее последующей детализации в других диаграммах;
4. Упростить взаимодействие между разработчиками, заказчиками и пользователями;

Спроектируем диаграмму вариантов использования для системы «Команда разработчиков». Актёрами станут: Заказчик, Менеджер, Разработчик и Система – основные действующие лица, где Заказчик предоставляет техническое задание (ТЗ) и оценивает счет, Менеджер рассматривает ТЗ и оформляет проект. Разработчик же отмечает количество часов, затраченных на работу над проектом.

Готовая диаграмма, спроектированная по описанию выше, находится в Приложении 1.

2. Разработка технического задания

Техническое задание – юридически значимый документ, содержащий исчерпывающую информацию, необходимую для постановки задач исполнителям на разработку, внедрение или интеграцию программного продукта.

Для данного проекта в качестве стандарта был выбран ГОСТ 19.201-78.

Техническое задание

1. Введение

1.1. Наименование программы

Наименование программы – информационная система «Команда разработчиков».

1.2. Краткая характеристика области применения

Система предназначена для организации взаимодействия между заказчиком, менеджером и разработчиком. Заказчик формирует техническое задание и согласовывает смету. Команда исполнителей оценивает объем работ и выставляет счет. Заказчик принимает решение о сотрудничестве.

2. Основания для разработки

Основанием для разработки является выполнение учебного курсового проекта. Наименование темы разработки – «Разработка системы «Команда разработчиков»».

3. Назначение разработки

Программа будет использоваться при взаимодействии заказчиков информационных систем и команд разработки проектов.

3.1. Функциональное назначение

Для заказчика программа предоставляет возможность быстрого создания ТЗ и оценки счета за работы, представленного командой разработки.

Для менеджера команды программа позволяет проанализировать ТЗ и выставить счет заказчику.

Для разработчика программа дает возможность оценить планируемый и фактический объем работ.

3.2. Эксплуатационное назначение

Программа должна эксплуатироваться при оценке ТЗ заказчика и объема работ, выполняемых командой. Программа должна запускаться на персональных устройствах, компьютерах, телефонах и ноутбуках, заказчика и команды исполнителей.

4. Требования к программе или программному изделию

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

После запуска программы должно открываться окно с формой ввода логина и пароля для входа.

В системе существует три типа пользователей: Заказчик, Менеджер и Разработчик.

После успешного входа открывается экран со списком проектов и фильтрами. На экране также расположены кнопки для открытия и создания проекта, создавать проекты может только Заказчик. Проекты могут иметь следующие статусы: Черновик, В работе, Требует Одобрения, Закончен, Закрыт, Заморожен.

При нажатии на проект должна открываться его карточка, разделенная на две части. Первая часть – ТЗ проекта, с возможностью редактирования в статусах Черновик и Требует Одобрения. Вторая часть – счет от команды исполнителей с итоговой суммой и списком необходимых работ. Каждый пункт в списке включает название, краткое описание, количество планируемых и фактических часов, стоимость. Во второй части экрана должен быть реализован функционал для одобрения изменений счета, предлагаемых командой разработчиков.

Для менеджера предоставляется такой же экран, как и для заказчика, но первая часть экрана с ТЗ имеет статичный вид, с функционалом одобрения изменений ТЗ, а вторая имеет возможность редактирования в статусе «В работе».

Здесь менеджер может добавлять работы в список и высчитывать их итоговую стоимость.

Для разработчика на экране карточки проекта редактируемыми являются только поля описания задачи, количества планируемых и фактически затраченных часов.

4.1.2. Требования к организации входных и выходных данных

Данные о проектах хранятся в базе данных. Данные сотрудников команды разработчиков в базу заполняет менеджер. Заказчик самостоятельно регистрируется в приложении и добавляет данные о своих ТЗ в базу данных через систему.

4.1.3. Требования к временным характеристикам

После изменения данных на стороне заказчика или команды исполнителей, данные отображаются на устройстве оппонента не позднее, чем через 5 секунд.

4.2. Требования к надежности

Вероятность безотказной работы системы должна составлять не менее 99% при условии исправности сети.

4.2.1. Требования к обеспечению надежного функционирования программы

В связи с тем, что в базе данных хранятся данные о договоренностях между заказчиком и исполнителями, базу данных необходимо резервировать.

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

1. Организация бесперебойного питания технических средств;
2. Использование лицензионного программного обеспечения.

4.2.2. Время восстановления после отказа

Время восстановления после отказа, вызванного сбоем электропитания технических средств, нефатальным сбоем операционной системы, не должно

превышать 10 мин при условии соблюдения условий эксплуатации технических и программных средств.

Время восстановления после отказа, вызванного неисправностью технических средств, фатальным сбоем операционной системы, не должно превышать времени, требуемого на устранение неисправностей технических средств и переустановки программных средств.

4.2.3. Отказы из-за некорректных действий

Отказы программы возможны вследствие некорректных действий оператора (пользователя) при взаимодействии с операционной системой. Во избежание возникновения отказов программы по указанной выше причине следует обеспечить работу пользователя без предоставления ему административных привилегий.

4.3. Условия эксплуатации

Программа запускается на персональных устройствах заказчика и команды исполнителей. База данных находится на стороннем компьютере. Должна существовать устойчивая связь по сети между клиентами и базой данных. Связь между базой и клиентами должна осуществляться через API.

4.3.1. Климатические условия эксплуатации

Требования к климатическим условиям не предъявляются.

4.3.2. Требования к видам обслуживания

Программа не требует проведения каких-либо видов обслуживания.

4.3.3. Требования к численности и квалификации персонала

Для установки приложения квалифицированный специалист не требуется, установка осуществляется по алгоритму самим пользователем.

4.4. Требования к составу и параметрам технических средств

Компьютер для установки информационно-справочной системы, включающий в себя:

1. Процессор с тактовой частотой, не менее 2400 МГц;

2. Оперативную память объемом, не менее 6 ГБ;

3. Видеокарту с объемом видеопамяти, не менее 2 ГБ;

4. Монитор;

5. Мышь;

6. Клавиатуру.

4.5. Требования к информационной совместимости

Требования к информационной и программной совместимости не предъявляются**.**

4.6. Требования к маркировке и упаковке

Программное обеспечение передается по сети Интернет и располагается на веб-хостинге GitHub с возможностью скачивания. Специальные требования к маркировке не предъявляются.

4.7. Требования к транспортировке и хранению

Требования к транспортированию и хранению не предъявляются.

4.8. Специальные требования

Программа должна обеспечивать взаимодействие с пользователем посредством графического пользовательского интерфейса, разработанного согласно рекомендациям компании-производителя операционной системы.

5. Требования к программной документации

Состав программной документации:

1. Техническое задание;
2. Диаграмма вариантов использования;
3. Диаграмма взаимодействия;
4. Диаграмма классов;

6. Технико-экономические показатели

Функциональность программы обусловлена учебным заданием и выведена за счет аналоговых приложений**.**

7. Стадии и этапы разработки

Разработка должна быть проведена в три стадии:

1. Техническое задание;

2. Технический и рабочий проекты;

3. Внедрение.

На стадии «Техническое задание» должен быть выполнен этап разработки, согласования и утверждения настоящего технического задания.

На стадии «Технический и рабочий проекты» должны быть выполнены перечисленные ниже этапы работ:

1. Разработка программы;

2. Разработка программной документации;

3. Испытания программы.

На стадии «Внедрение» должен быть выполнен этап разработки «Подготовка и передача программы».

8. Порядок контроля и приемки

Приемо-сдаточные испытания программы осуществляются на базе учебного заведения путем проверки проделанной работы.

3. Разработка диаграммы деятельности

Диаграмма деятельности – диаграмма UML, которая демонстрирует логику и последовательность действий при выполнении программы.

На рисунке 1.1 представлена диаграмма деятельности для выставления начального счета по анализу ТЗ в системе.

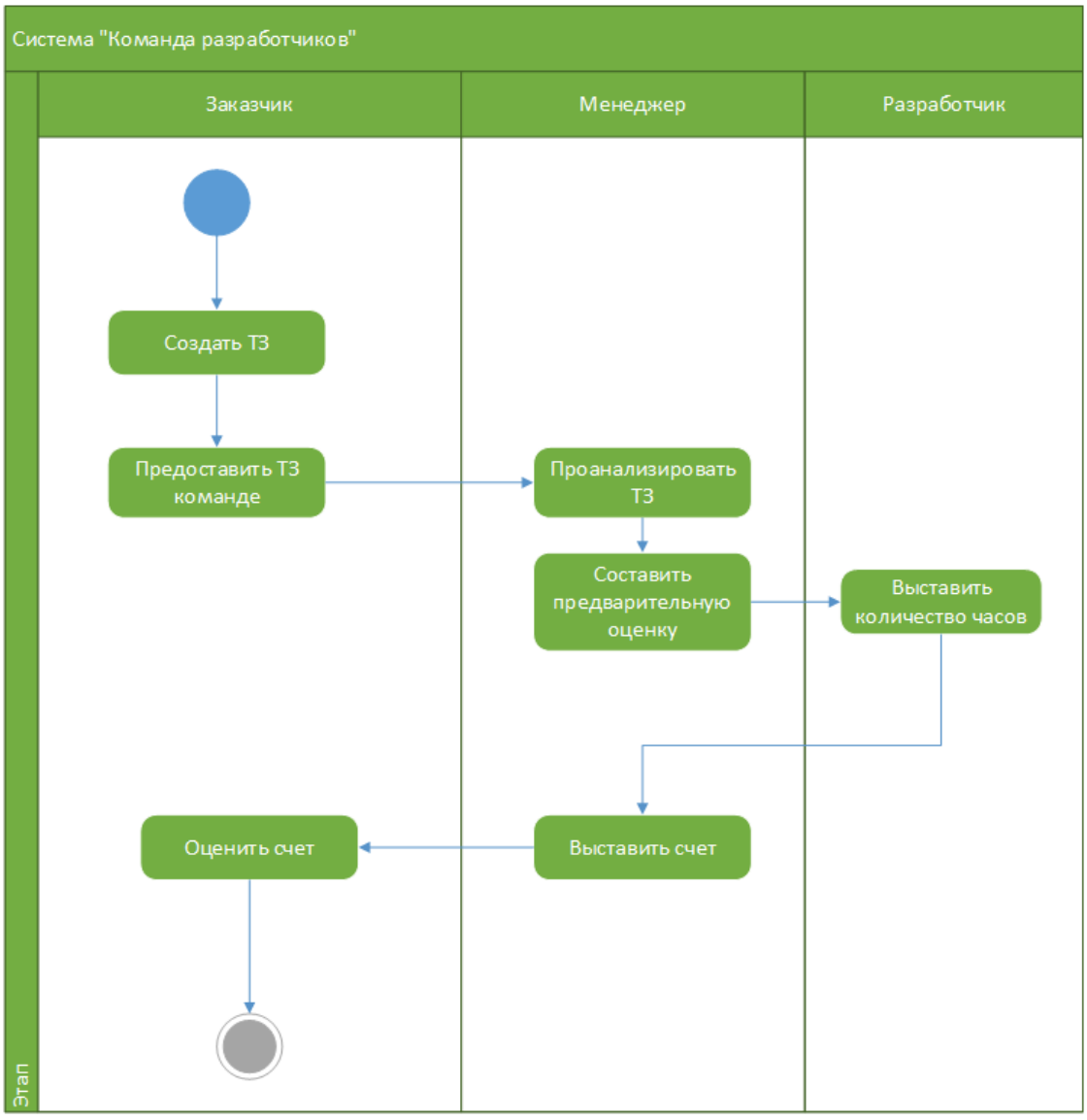


Рис.1.1. Диаграмма деятельности

4. Разработка диаграммы взаимодействий

Диаграмма взаимодействия предназначена для описания взаимодействия между элементами программы и передачи сообщений между ними.

На рисунке 2.1 представлена диаграмма принятия и одобрения изменений счета в процессе работы над проектом.

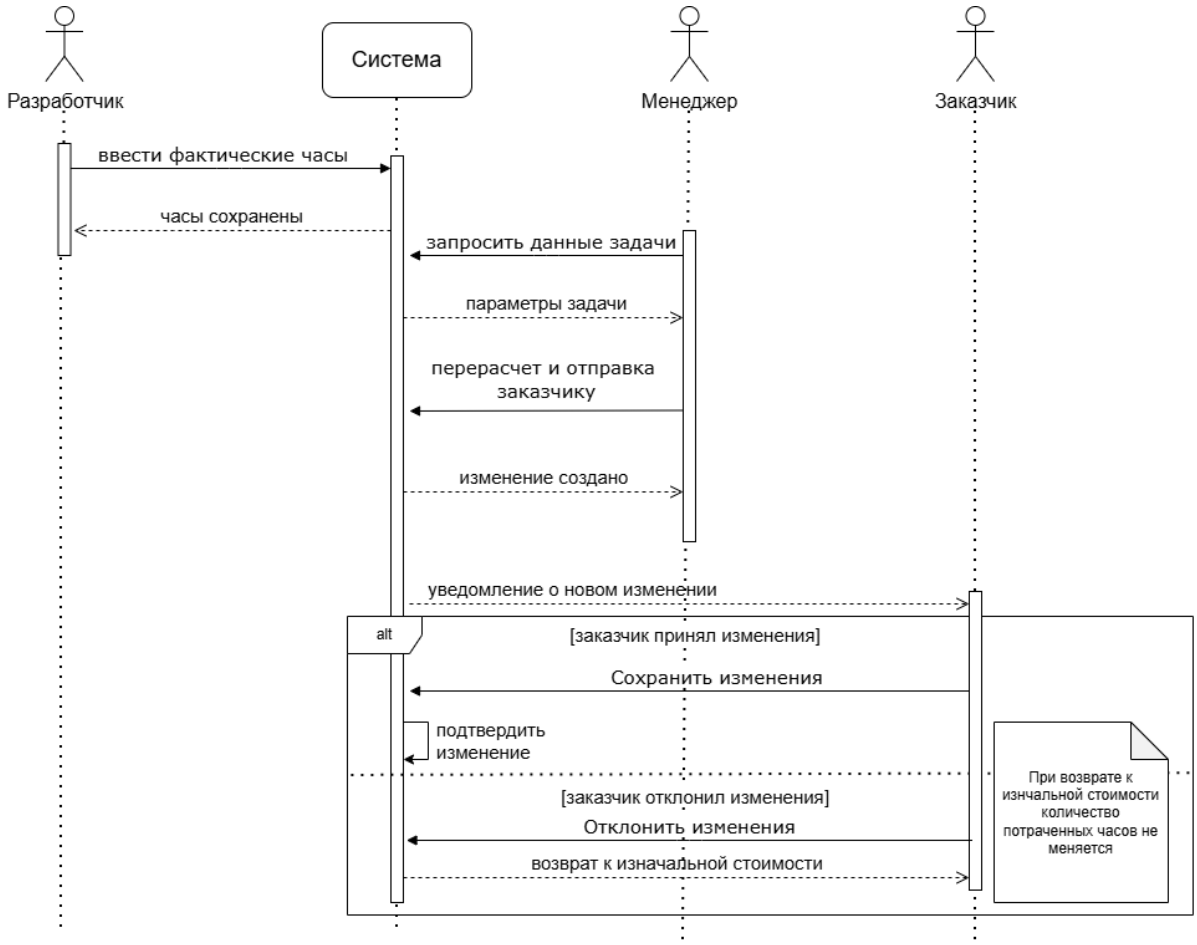


Рис. 2.1. Диаграмма взаимодействия

5. Модели и генерирование кода на основе UML-диаграмм

Следующим этапом необходимо определить модели классов, которые будут использованы в системе.

Для этого спроектируем диаграмму классов и выделим следующие сущности:

1. Компания – сущность, представляющая организацию. Содержит идентификатор и название.
2. Команда – сущность, объединяющая разработчиков. Содержит идентификатор, название, описание.
3. Пользователь – основная сущность для авторизации и распределения ролей. Содержит идентификатор, имя, фамилию, роль, логин, хэш пароля, краткое описание.
4. Проект – сущность, объединяющая заказчика и команду исполнителей. Содержит идентификатор, название, статус, описание.
5. Техническое задание – документ, описывающий требования к проекту. Содержит идентификатор, название, сроки проекта, описание.
6. Пункты ТЗ – детализация технического задания. Содержит идентификатор, название, описание.
7. Счет – финансовый документ. Содержит идентификатор, название, стоимость, даты начала и окончания работ.
8. Список задач – этапы работ в рамках счета. Содержит идентификатор, название, процент выполнения.
9. Задача – конкретная единица работы. Содержит идентификатор, название, описание, статус, количество планируемых и фактических часов, стоимость.
10. Изменения для подтверждения – сущность, фиксирующая изменения в ТЗ или счетах. Содержит идентификатор, название, описание, время создания, статус.

Создадим связи между классами, руководствуясь правилами реляционных баз данных:

Ассоциация связывает Пользователя с Проектами как заказчика, Пользователя с Задачами как разработчика, а также Пользователя с Изменениями как создателя или согласующего. Также с помощью ассоциации связаны Проект и Команда, Задача и Список задач.

Агрегация используется для связи Счета и Списка задач, а также ТЗ и Пунктов ТЗ, Пользователей и Команд.

Композиция применяется там, где дочерняя сущность не может существовать без родительской. Например, Пункты ТЗ не могут существовать без самого ТЗ. При удалении родительского объекта дочерние также удаляются, что в базе данных реализовано через каскадное удаление.

Множественность показывает, сколько объектов участвует в связи. Например, одного Пользователя может быть много Задач. При этом у одного Пункта ТЗ может быть несколько связанных Изменений, а изменение ссылается либо на задачу, либо на счет.

Спроектированная диаграмма классов представлена в Приложении 2.

6. Оптимизация программного кода

Следующим этапом необходимо реализовать классы в программном коде, а точнее заключить их в пространства имен. Пространства имен позволяют организовать код программы в логические блоки, объединить и отделить от остального кода некоторую функциональность, которая связана одной идеей или выполняет некоторую задачу.

По очереди сгенерируем каждый из необходимых классов в соответствии с их функциями, определим области видимости для отдельных элементов.

Первый класс, класс Пользователей, рассмотрен на рисунке 3.1.

```
using System;
using System.Collections.Generic;

namespace ParamonovAPI.Models;

public partial class User
{
    public int IdUser { get; set; }

    public string FirstName { get; set; } = null!;

    public string LastName { get; set; } = null!;

    public string Login { get; set; } = null!;

    public string PasswordHash { get; set; } = null!;

    public int? IdTeam { get; set; }

    public int? IdCompany { get; set; }

    public string? ShortDescription { get; set; }

    public virtual ICollection<ChangeRequest> ChangeRequestIdApproverNavigations { get; set; } = new List<ChangeRequest>();

    public virtual ICollection<ChangeRequest> ChangeRequestIdCreatorNavigations { get; set; } = new List<ChangeRequest>();

    public virtual Company? IdCompanyNavigation { get; set; }

    public virtual Team? IdTeamNavigation { get; set; }

    public virtual ICollection<Project> Projects { get; set; } = new List<Project>();

    public virtual ICollection<Task> Tasks { get; set; } = new List<Task>();
}
```

Рис. 3.1. Класс Пользователей

В классе ChangeRequest (изменения для подтверждения) особое внимание уделяется свойствам, которые могут принимать значение null. Например,

IdApprover может быть пустым, пока изменение находится в статусе «Создано» (Создатель есть, а согласующий еще не назначен). Это реализуется с помощью nullable-типов (int?). Класс отражен на рисунке 3.2.

```
1   using System;
2   using System.Collections.Generic;
3
4   namespace ParamonovAPI.Models;
5
6   public partial class ChangeRequest
7   {
8       public int IdChange { get; set; }
9
10      public int IdSpecItem { get; set; }
11
12      public int? IdTask { get; set; }
13
14      public int? IdInvoice { get; set; }
15
16      public string Name { get; set; } = null!;
17
18      public string? Description { get; set; }
19
20      public DateTime CreatedAt { get; set; }
21
22      public int IdCreator { get; set; }
23
24      public int? IdApprover { get; set; }
25
26      public virtual User? IdApproverNavigation { get; set; }
27
28      public virtual User IdCreatorNavigation { get; set; } = null!;
29
30      public virtual Invoice? IdInvoiceNavigation { get; set; }
31
32      public virtual SpecificationItem IdSpecItemNavigation { get; set; } = null!;
33
34      public virtual Task? IdTaskNavigation { get; set; }
35  }
```

Рис. 3.2. Класс Изменений

В классе Task (задача) реализованы свойства для учета планируемых и фактических часов. В отличие от оригинальных сгенерированных классов с throw new NotImplementedException();, мы используем автоматические свойства. Для

связи с разработчиком используется внешний ключ IdDeveloper и навигационное свойство IdDeveloperNavigation (см. рис. 3.3.).

```
using System;
using System.Collections.Generic;

namespace ParamonovAPI.Models;

public partial class Task
{
    public int IdTask { get; set; }

    public int IdTaskList { get; set; }

    public string? Description { get; set; }

    public decimal? PlannedHours { get; set; }

    public decimal? ActualHours { get; set; }

    public decimal? Cost { get; set; }

    public int? IdDeveloper { get; set; }

    public virtual ICollection<ChangeRequest> ChangeRequests { get; set; } = new List<ChangeRequest>();

    public virtual User? IdDeveloperNavigation { get; set; }

    public virtual TaskList IdTaskListNavigation { get; set; } = null!;
}
```

Рис. 3.3. Класс Задач

Таким образом, в ходе оптимизации программного кода были реализованы все классы-сущности. Каждое свойство имеет корректный тип данных (строки, целые числа, даты, десятичные дроби), а также модификаторы доступа public. Использование коллекций ICollection<T> и виртуальных свойств (virtual) позволяет Entity Framework эффективно выполнять ленивую загрузку (Lazy Loading) связанных данных и строить графы объектов при запросах к базе данных. Сгенерированный код полностью соответствует спроектированной ранее диаграмме классов и готов к использованию в слое доступа к данным.

7. Разработка программного обеспечения для проектируемой предметной области

Разработка приложения началась с базы данных PostgreSQL, были созданы таблицы по образцу спроектированной диаграммы классов. Для нескольких полей использовались перечисления: статус проекта, статус задачи, статус запроса на изменение и роль пользователя описаны через типы ENUM. Это решение упростило контроль допустимых значений на уровне самой СУБД.

Серверная часть выполнена на ASP.NET Core. Серверная часть осуществляет обмен данными между клиентским приложением и базой данных, в ней реализован механизм регистрации новых пользователей и аутентификация, построенная на JWT-токенах. Также в серверной части работает алгоритм хэширования паролей.

Настольное приложение выполнено на WPF по паттерну MVVM. Слой доступа к API вынесен в отдельные сервисы с общим базовым классом.

Экран входа содержит поля логина и пароля и кнопку перехода к регистрации (см. рис. 4.1).

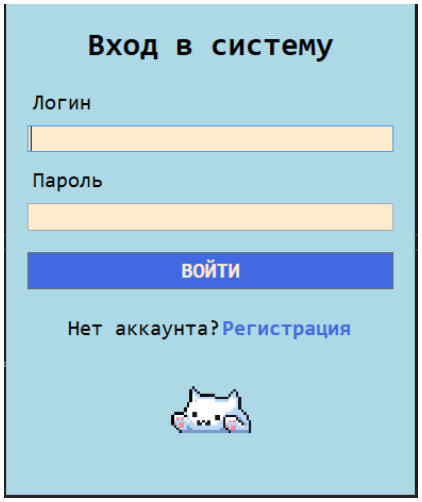


Рис. 4.1. Экран входа в систему

При заполнении неверных данных система выводит информационное сообщение (см. рис. 4.2).

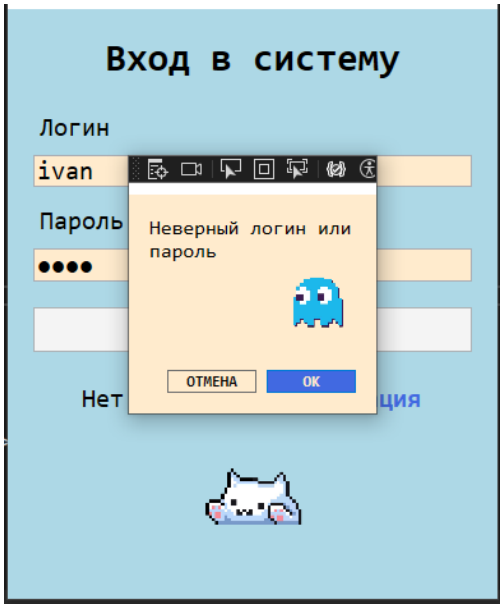


Рис. 4.2. Экран входа в систему

Из экрана входа можно перейти на экран регистрации (см. рис. 4.3).

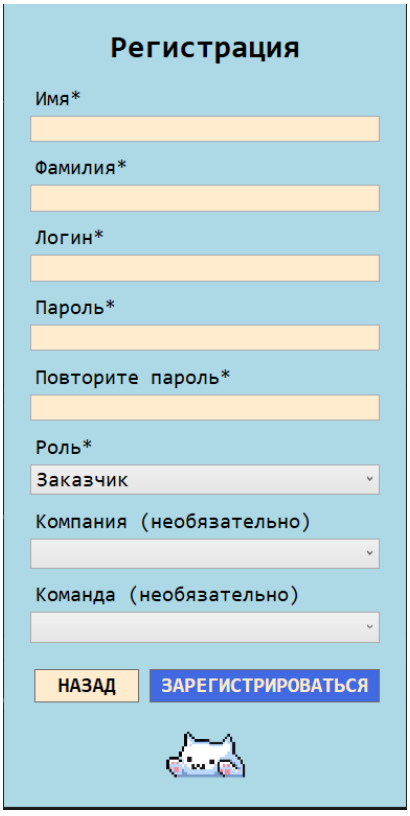


Рис. 4.3. Экран входа в систему

После успешной аутентификации открывается окно со списком проектов (см. рис. 4.4.). В верхней части расположены кнопки фильтрации по статусам. Кнопка создания проекта видна только заказчику. Список проектов показывает название, описание, статус и общую стоимость. Кнопка открытия активна только при выбранном проекте.

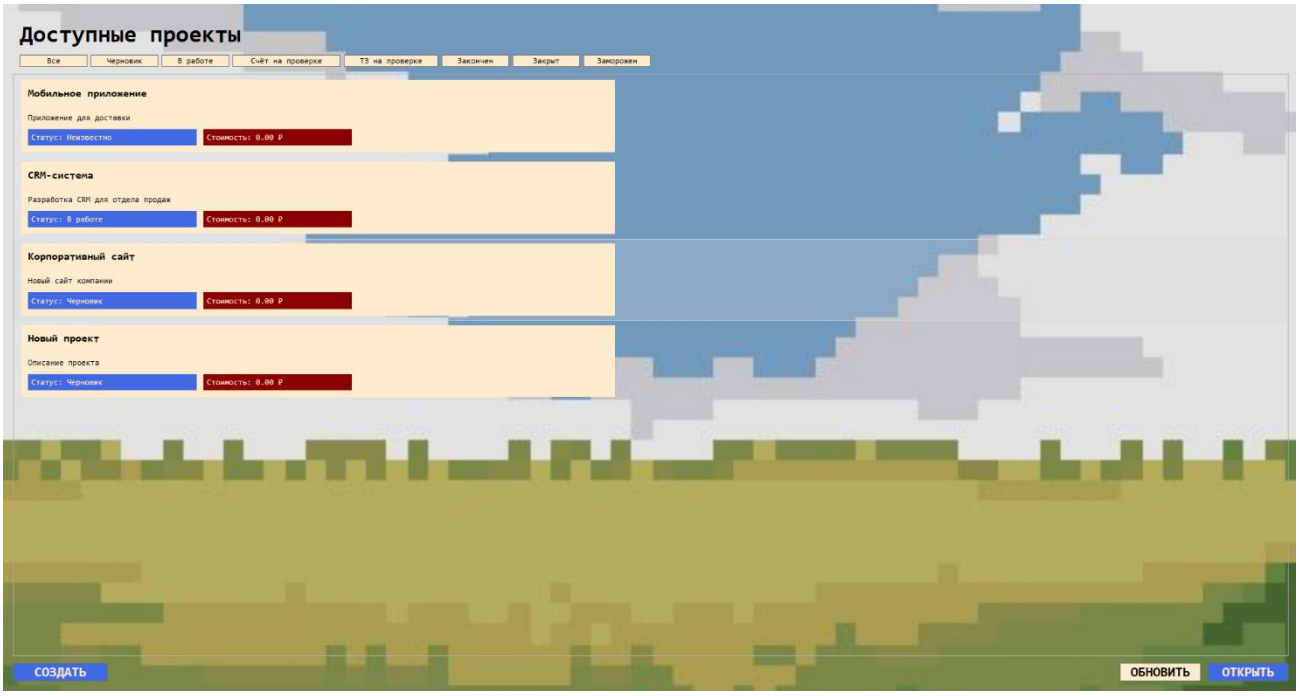


Рис. 4.4. Экран со списком проектов

При открытии или создании проекта открывается экран карточки проекта, разделенный на две области (см. рис. 4.5.). Слева расположено техническое задание с пунктами. Справа размещён счёт с задачами и итоговой суммой. В шапке отображаются заказчик, его компания, команда и её компания.

Видимость и доступность элементов зависят от роли пользователя и статуса проекта. Заказчик редактирует ТЗ в черновике и назначает команду. После отправки проекта команде редактирование блокируется. Менеджер заполняет счёт

и задачи. Разработчик в этом же статусе вносит фактические часы. В статусе «На одобрении» заказчик принимает или отклоняет изменения и может править ТЗ.

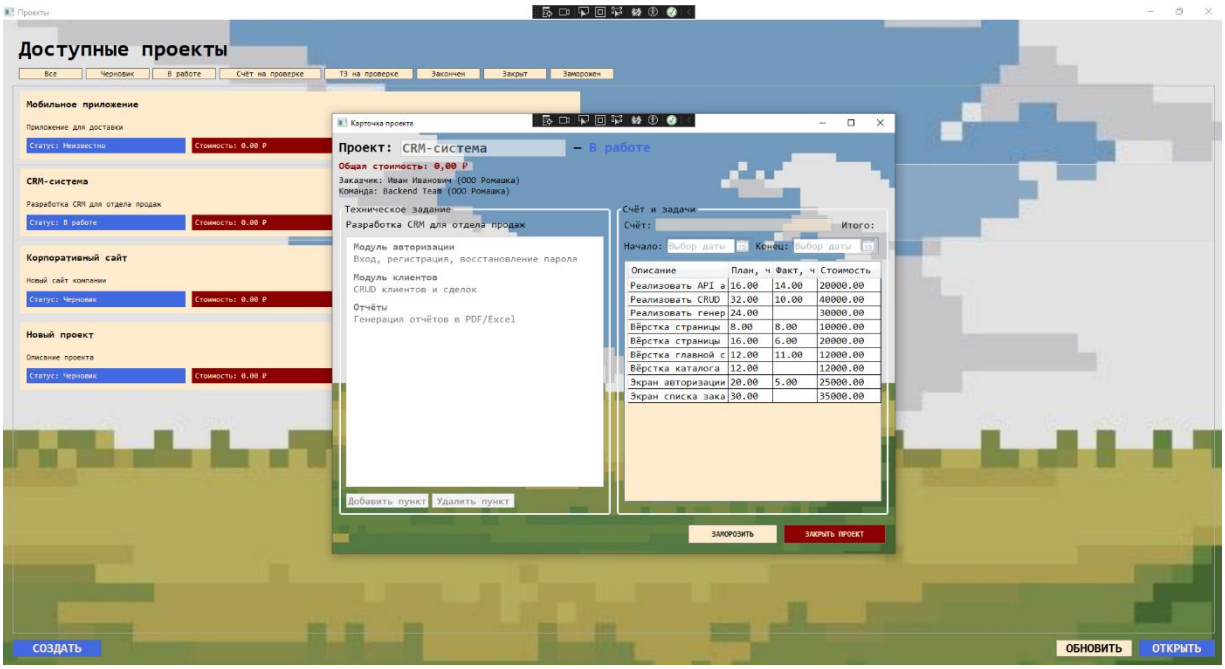


Рис. 4.5. Экран карточки проекта

Для менеджера карточка проекта будет выглядеть следующим образом:

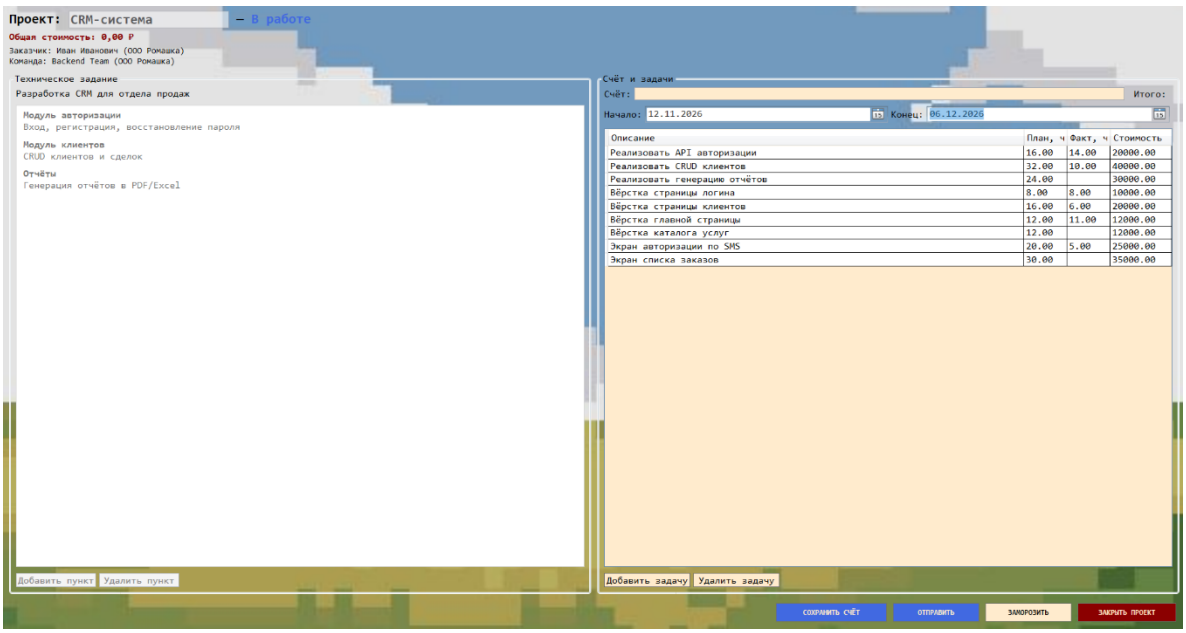


Рис. 4.5. Экран карточки проекта

Поскольку проект находится в статусе «В работе», менеджер может редактировать название счета, сроки выполнения проекта, а также добавлять и удалять задачи (см. рис. 4.6.).

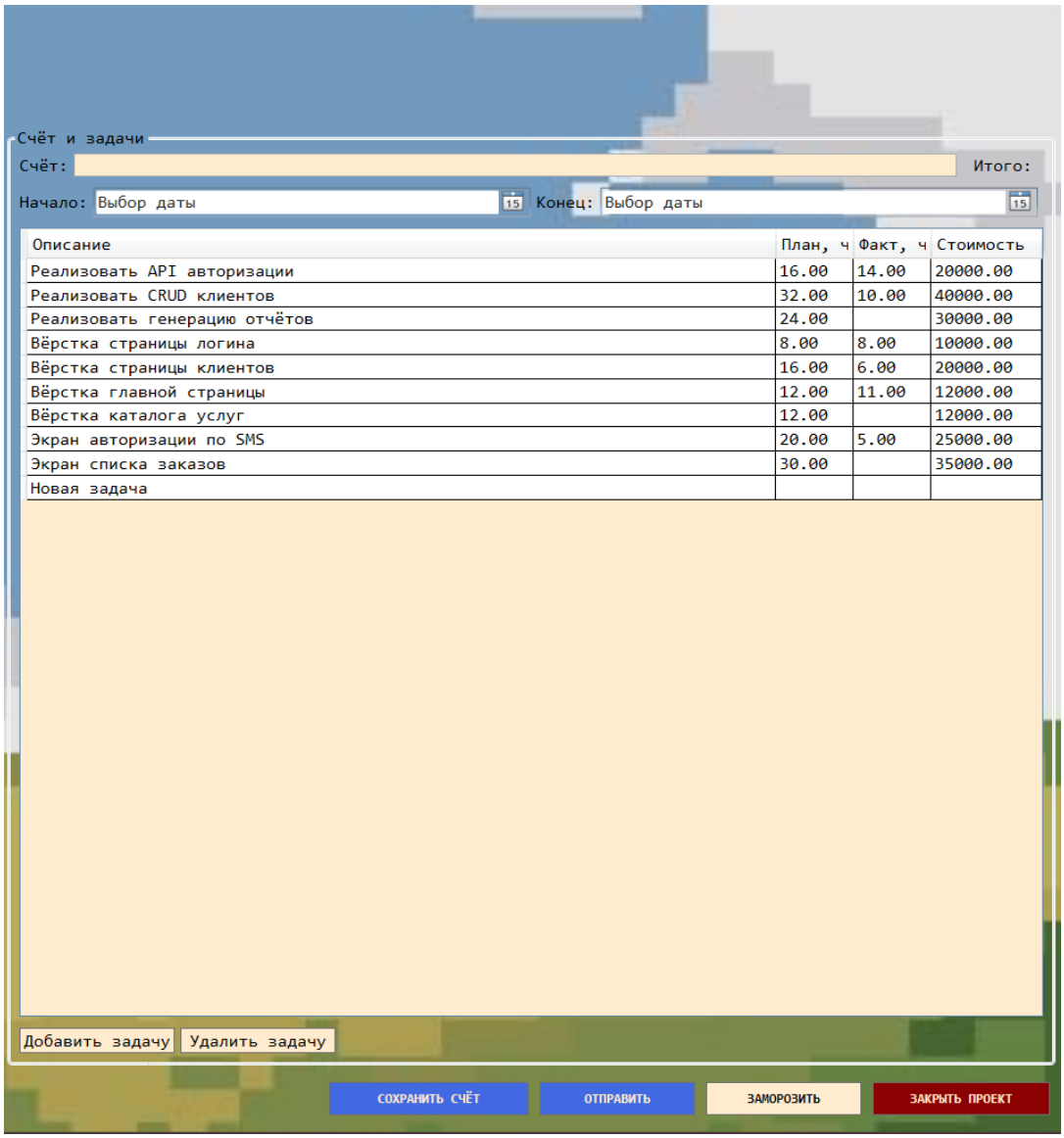


Рис. 4.6. Редактирование списка задач

8. Работа с системой контроля версий

Система контроля версий фиксирует историю изменений исходного кода, это позволяет вернуться к предыдущей рабочей версии, если новая содержит ошибки.

Во время работы над проектом использование Git и GitHub решает несколько задач:

1. GitHub выступает в роли удаленного резервного хранилища. Код проекта не привязан к одному компьютеру, при поломке оборудования или случайном удалении файлов репозиторий можно восстановить из удаленной копии.
2. Git записывает и сохраняет историю изменений, всегда есть возможность вернуться к старой версии.
3. Несколько человек могут одновременно писать один и тот же проект. Git помогает объединять их правки и разрешать конфликты, если два разработчика изменили одну строчку кода.
4. Файл readme.md в корне репозитория содержит описание проекта. В нем указываются назначение системы, инструкция по запуску и сведения об авторе, это делает проект понятным для стороннего читателя.

Итоговый проект расположен в репозитории GitHub, доступном по ссылке https://github.com/Svetlana184/ParamonovKursach.git. Репозиторий содержит полные текстовые материалы, описание программы и ссылки на все составляющие.

Заключение

Таким образом, в ходе курсового проекта была спроектирована и разработана информационная система «Команда разработчиков».

Был проведен тщательный анализ, в ходе которого были спроектированы диаграммы вариантов использования, диаграмма классов и диаграмма взаимодействия. Также было сформулировано техническое задание к проекту.

После этого на основе спроектированных диаграмм был создан программный продукт, включающий в себя базу данных, сервис API и настольное приложение.

Получившийся в результате курсового проекта продукт соответствует требованиям технического задания и позволяет заказчикам информационных систем быстро и легко взаимодействовать с командой исполнителей, моментально вносить правки и корректировки в проект.

Список использованных источников

1. Лысенкова С.Н. Основы проектирования баз данных: учебно- методическое пособие. 2019. URL: https://e.lanbook.com/book/133118 (дата обращения: 09.11.2026).
2. Московский физико-технический институт, НОУ ИНТУИТ курс «Визуальное моделирование: теория и практика». URL: https://intuit.ru/studies/professional_skill_improvements/1467/info (дата обращения: 09.11.2026).
3. Официальная документация PostgreSQL. URL: https://www.postgresql.org/docs/ (дата обращения: 10.11.2026).
4. Практические руководства по C#, ASP.NET Core и базам данных – Metanit.com. URL: https://metanit.com/sharp/ (дата обращения: 10.11.2026).
5. Статьи по проектированию API, настройке PostgreSQL и паттернам WPF – Habr.com. URL: https://habr.com/ru/ (дата обращения: 10.11.2026).
6. Официальная документация Microsoft ASP.NET Core. URL: https://learn.microsoft.com/ru-ru/aspnet/core/ (дата обращения: 10.11.2026).
7. Официальная документация Microsoft .NET (WPF). URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ (дата обращения: 10.11.2026).

Список сокращений

API – Application Programming Interface (программный интерфейс приложения)

MVVM – Model-View-ViewModel

WPF – Windows Presentation Foundation

UML – Unified Modeling Language (унифицированный язык моделирования)

БД – база данных

МДК – междисциплинарный курс

СУБД – система управления базами данных

ТЗ – техническое задание

Приложение 1

Диаграмма вариантов использования системы «Команда разработчиков»



Рис. 6 Диаграмма вариантов использования

Приложение 2

Диаграмма классов системы

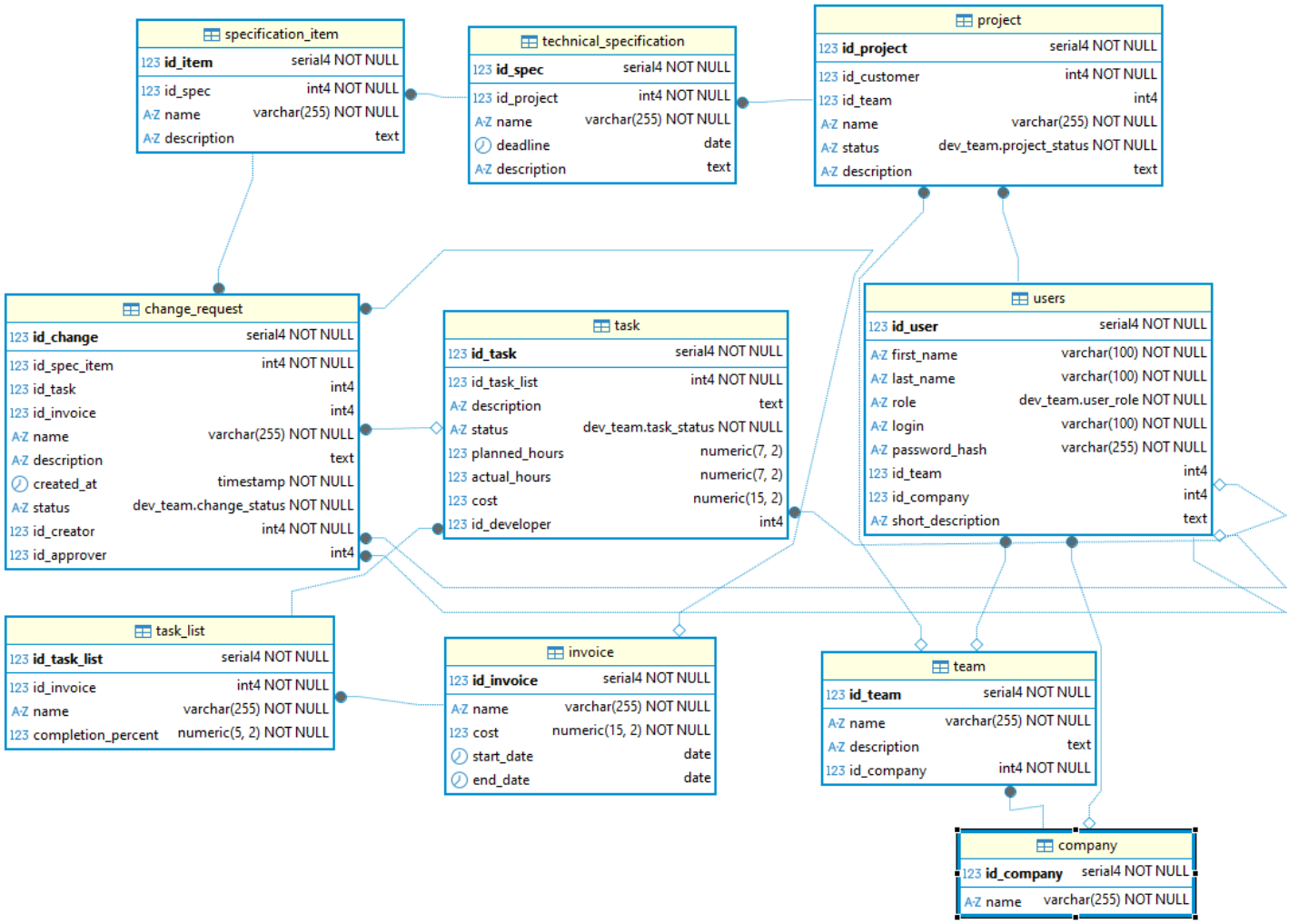


Рис. 7 Диаграмма классов